



CS6502 – Applied Big Data and Visualization

Project report

Large-Scale Traffic Volume Prediction on Irish National Roads

Team - 21

Krithikaa Jothi Prakash - 25287591

Hari Gokul Ganesan Kalavathi - 25287613

Sidharthan Jayavelu - 25110004

Nimra Zia - 25009877

Sheetal Padmanabhan - 25022911

Ishitha Prakashan - 25181483

Department of Computer Science and Information Systems
Faculty of Science and Engineering
University of Limerick

Table of Contents	Pg No
Abstract	4
1. Introduction	4
2. Project Objective and End-to-End Workflow	4
Table 1. Notebook-by-Notebook pipeline map	5
3. Related Work	5
4. Research Methodology	5
4.1 Dataset Selection	5
Table 2. Dataset Summary	6
5. Data Ingestion and Cleaning	7
Table 3. Cleaning summary	7
6. Feature Engineering and Encoding	8
7. Feature Analysis	8
Table 4. Final feature groups	10
8. Exploratory Data Analysis	10
9. Evaluation - Baseline Regression Modelling	11
Table 5. Baseline model comparison on the temporal test set	11
10. Hyperparameter Tuning and Optimisation	12
Table 6. Final Model Evaluation and Interpretation	12
11. Results – Final Model Evaluation and Interpretation	13
12. Discussion	14
13. Limitations and Future Work	15

14. Conclusion	15
15. References	15

Abstract

This report has the details of the complete design, implementation and evaluation of a large traffic volume prediction system built on Apache Spark and deployed on Databricks. The dataset includes 7,579,711 hourly traffic records collected from Irish national road sensors throughout 2019 organized into four quarterly tables for Q1, Q2, Q3 and Q4.

The project focuses on the regression task of predicting hourly vehicle counts (`vehicle_count`) using 19 features derived from sensor measurements, time attributes, location identifiers and created categorical variables. The pipeline includes data ingestion and cleaning, outlier detection through K-means anomaly scoring, feature engineering, selection based on correlation and model performance, comparison of baseline models across five SparkML algorithms and tuning of hyperparameters for the best model.

While considering the five baseline models which are Linear Regression, Generalised Linear Regression, Decision Tree, Random Forest and Gradient Boosted Tree (GBT), GBT shows the best performance with $R^2 = 0.9926$ and $RMSE = 30.63$. After further tuning ($maxDepth = 10$, $maxIter = 50$, $stepSize = 0.1$) the performance improved to $R^2 = 0.9936$ and $RMSE = 28.49$ that represents a 9.5% reduction in RMSE compared to the baseline. Analysis of feature importance showed `avg_gap`, `avg_headway`, `congestion_index_index` and `hour` were the main predictors of hourly traffic volume.

1. Introduction

Traffic prediction is a major issue in the modern-day transport control. Traffic prediction models assist in managing congestion in real time, planning the infrastructures, providing the dynamic routing information and making transport policies. With sensor networks expanding along national and regional roads there is now access to substantial amounts of high-resolution traffic data but utilizing this data effectively requires distributed computing frameworks that can handle millions of records efficiently.

This project uses Apache Spark to create a scalable machine learning pipeline for traffic volume prediction on Irish national roads. The dataset obtained from the Transport Infrastructure Ireland (TII) sensor networks covers the entire year of 2019 at an hourly resolution across various sensor sites and lanes. The primary goal is to predict the total vehicle count for a specific hour based on traffic measurement features for that one hour, sensor location and lane.

The project follows the pipeline in the module requirements: data ingestion and cleaning using SparkSQL and Spark Data frames, exploratory data analysis, feature engineering and selection, training five SparkML regression models and optional hyperparameter tuning through Gradient Boosted Tree optimization. This dataset was chosen due to its large size over 7.5 million records, rich features and clear relevance to transportation analytics.

2. Project Objective and End-to-End Workflow

The main aim of this project is to build a reliable Spark-based regression pipeline that can both explain and predict hourly traffic volume while still being realistic for real-world use.

The project began by converting raw per-vehicle sensor data into hourly traffic tables. After that, the quarterly datasets were loaded and combined into one unified Spark DataFrame. The dataset was then prepared by handling null values, checking duplicates, and removing anomalous records.

Once the data was cleaned, new categorical features were created to better represent traffic behaviour and time-based patterns. Exploratory data analysis and feature analysis were then carried out to understand the relationships in the data and to identify any possible leakage.

Finally, multiple Spark ML regression models are trained and compared using a temporal train-test split. The strongest baseline model is then tuned further, and the final model is interpreted using residual analysis and feature-importance results.

Table 1. Notebook-by-notebook pipeline map

Notebook	Main purpose	Spark emphasis
1	Raw data download and hourly aggregation	SparkSession, distributed aggregation logic
2	Data ingestion, cleaning, and feature engineering	Spark tables, DataFrames, SparkML K-Means
3	Feature analysis and selection	Correlation analysis, leakage detection, deployable feature set
4	EDA on final dataset	Spark summaries, grouped statistics, scalable visual preparation
5	Baseline modelling	VectorAssembler, temporal split, multiple Spark regressors
6	Hyperparameter tuning	TrainValidationSplit and parameter grid search
7	Final model evaluation	Final tuned GBT, residual analysis, feature importance

3. Related Work

Traffic volume prediction has been studied using both traditional statistical methods and machine learning approaches. Simpler models such as Linear Regression are useful as baselines, but they often struggle to capture the non-linear and interaction-heavy nature of traffic data. More recent work commonly uses tree-based methods such as Decision Trees, Random Forests, and Gradient Boosted Trees because they can better model complex relationships between time, location, and traffic-state variables.

From a data-engineering perspective, large-scale traffic datasets are also closely connected to big data research. When traffic data is too large or too fine-grained for local analysis, distributed frameworks such as Hadoop and Apache Spark are commonly used. In comparison with disk-based batch processing, Spark is better suited to iterative analytics and machine learning because of its DataFrame abstractions, SparkSQL capabilities, and in-memory execution model. This project follows that broader direction by applying Spark not only to model development, but also to raw-data aggregation, data cleaning, feature engineering, and tuning. Relative to simpler traffic prediction studies, the present work places greater emphasis on scalable data preparation and leakage-aware model design.

4. Research Methodology

4.1 Dataset Selection

Notebook 1, titled Traffic Count Data Aggregator, explains where the dataset used in this report comes from. The raw source is the Transport Infrastructure Ireland (TII) Traffic Count Data repository, which publishes daily per-vehicle CSV files using the URL pattern <https://data.tii.ie/Datasets/TrafficCountData/yyyy/MM/dd/per-vehicle-records-yyyy-MM-dd.csv>. Each file

contains one record per detected vehicle rather than a pre-aggregated hourly summary, which means the raw source is granular, high-volume, and operational in character.

In practical terms, the raw data is too detailed to be used directly for modeling. Since the goal is to predict hourly traffic volume, the model needs one record for each sensor, lane, and hour, rather than one record for every individual vehicle. Notebook 1 serves as the link between the public raw data and the structured quarterly datasets used later in the project. It automatically downloads the daily files, processes them in Spark, and gradually builds the aggregated quarterly datasets for Q1 to Q4 2019.

The aggregation logic used in the project is simple in structure but very important. Each downloaded CSV file is first loaded into Spark with schema inference enabled, and then the data is grouped by **cosit, lane, year, month, day, and hour**. Together, these fields define the final unit of analysis: traffic observed at a particular sensor, in a particular lane, during a particular hour. For each group, the aggregator calculates **vehicle_count** as the total number of records also generated summary measures for speed, vehicle length, headway, and gap using functions such as average, maximum, minimum, and standard deviation.

The notebook iterates from **1 January 2019 to 31 December 2019**, downloads each available daily file, aggregates it immediately, appends the result to the relevant quarterly CSV, and deletes the temporary raw download. This step-by-step approach is efficient because it avoids storing an entire year of raw per-vehicle files locally while still building a complete dataset for the full year. By the end of Notebook 1, the raw daily sensor records have been transformed into four quarterly hourly tables that are ready for Spark SQL analysis, feature engineering, and large-scale machine learning.

This scale is important for two main reasons. First, the dataset is too large for repeated local or spreadsheet-based analysis, so it benefits Spark's distributed processing capabilities. Second, the pipeline reflects an important big data principle highlighted in the module: before machine learning can begin, meaningful analytical datasets often need to be built from raw operational data through distributed ingestion and transformation.

Table 2. Dataset summary

Property	Value
Source granularity	Per-vehicle traffic observations aggregated to hourly level
Geographical scope	Irish national road sensor network
Temporal coverage	January to December 2019
Unified raw rows	7,579,711
Rows after cleaning	7,503,913
Final modelling columns	20 total (19 input features + vehicle_count target)
Storage format in workflow	Databricks metastore tables consumed through Spark DataFrames
Prediction task	Regression: hourly vehicle_count

5. Data Ingestion and Cleaning

Notebook 2 begins by loading the four quarterly tables directly from the Databricks metastore into Spark DataFrames and then combining them into one large analytical table. The Spark DataFrames supports the

inspection of schemas, columnar operations, distributed filtering, and the clean integration with SparkML in the future of the pipeline.

The first schema entails 29 columns such as sensor identifiers, times attributes, number of traffic classes, and summary measures such as speed, gap, headway, and vehicle length. The cleaning is then done in three major stages. First, duplicate checking was done to ensure that there were no duplicate rows. Second, there were four columns of standard-deviation - speed_std, length_std, headway_std, and gap_std - with 107,850 null values. These nulls were not accidental missing data, but rather where there was only one vehicle observation in one hourly interval, so the variability was undefined. It is statistically and operationally convenient, therefore, to replace them with 0.

Third, SparkML K-Means anomaly scoring is used to eliminate anomalous rows in the project. Each record is then given a score based on its distance to the closest cluster centre after the assembly and standardisation of vectors. An 99th-percentile cutoff of 8.9896 isolates extreme cases. This eliminates 75,798 rows, and the downstream analysis is done on 7,503,913 rows. The process is useful as it employs a scalable ML-based method as opposed to manual hard-coded thresholds, which is appropriate to the big data nature of the task.

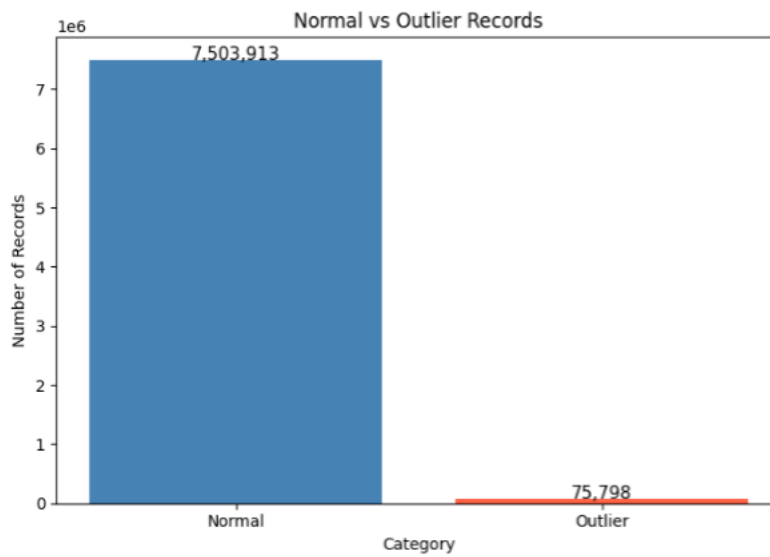


Figure 1. Normal versus outlier records after K-Means anomaly scoring. Only about 1 percent of records were removed, preserving the overwhelming majority of the dataset while filtering extreme behaviour

Table 3. Cleaning summary

Step	Finding	Action / Result
Deduplication	No duplicate rows detected	Row count unchanged at 7,579,711
Null handling	107,850 nulls in each variability column	Imputed with 0 to represent zero within-hour variability for single-vehicle cases
Outlier removal	Anomaly threshold = 8.9896 from K-Means scores	75,798 rows removed; 7,503,913 rows retained

6. Feature Engineering and Encoding

Once cleaned, the project engineers discretize categorical features to enhance interpretability and give more modelling cues. Time-of-day bands are computed based on hour, day type differentiates weekday and weekend behaviour, traffic level groups traffic intensity, speed category bins average speed and congestion index which

is a combination of headway and gap information into a form of interpretable congestion state. Since Spark ML estimators take numerical data as input, categorical engineered features are transformed into indexed ones with StringIndexer as part of a Pipeline. This maintains the distributed workflow and does not export the data into a different local preprocessing environment. An important modelling concept can be seen in Notebook 3: not all of the fields available should be utilized just because they are available. Counts of variables like CAR count, LGV count or other counts of vehicle classes are highly correlated with vehicle count since they are part and parcel of the target. Their inclusion would cause leakage in the targets and bloat evaluation numbers without generating a realistic predictive system.

7. Feature Analysis

Notebook 3 performs structured feature analysis in three parts: feature-target correlation, feature-feature multicollinearity inspection, and variance-based screening. The most important output is the transition from a broad candidate set to a deployable final feature set that is predictive without leaking the answer.

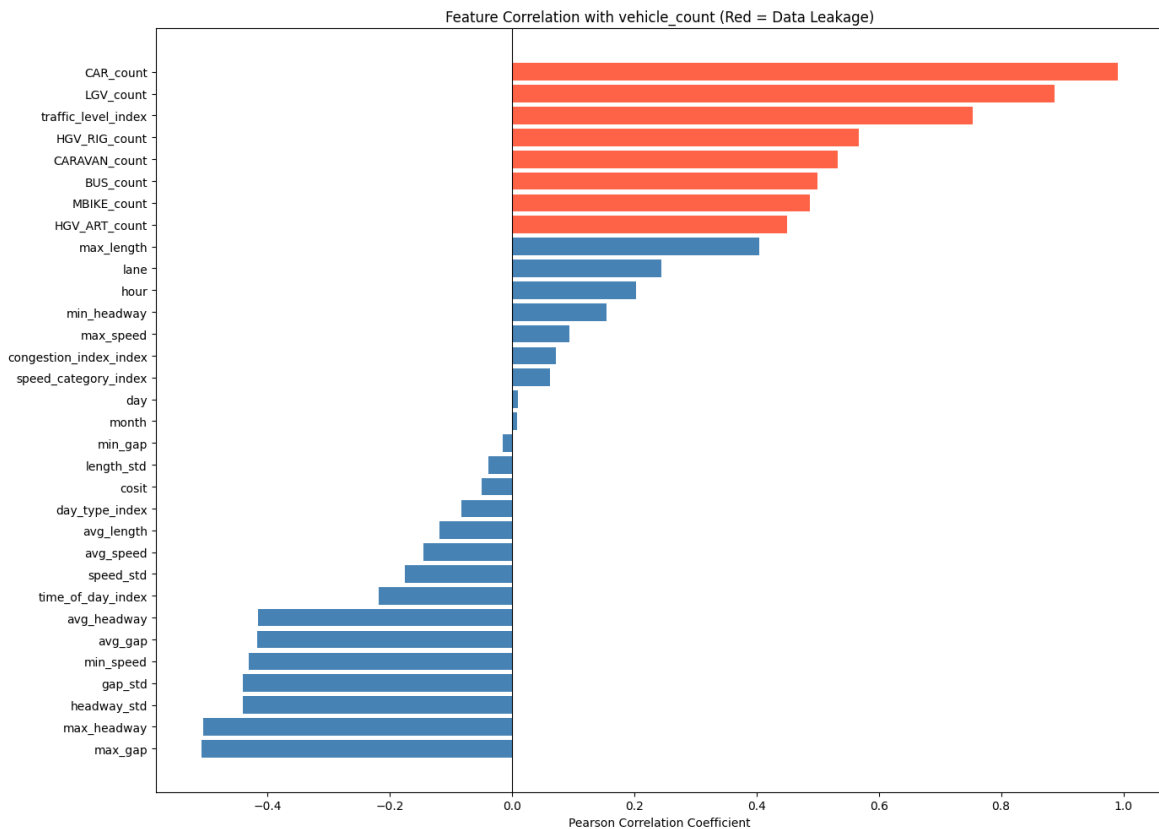


Figure 2. Correlation with vehicle_count. Red bars indicate leakage-prone variables that either directly compose the target or reveal near-target information and are therefore excluded from training.

A complementary feature-feature correlation heatmap was used to inspect multicollinearity inside the retained candidate set. The clearest pattern is a dense cluster among headway and gap statistics, where several relationships are extremely strong, while hour and time_of_day_index are almost perfectly inversely related because the latter is derived from hour. This confirms that Notebook 3 is not only removing leakage, but also controlling redundancy before Spark ML model training.

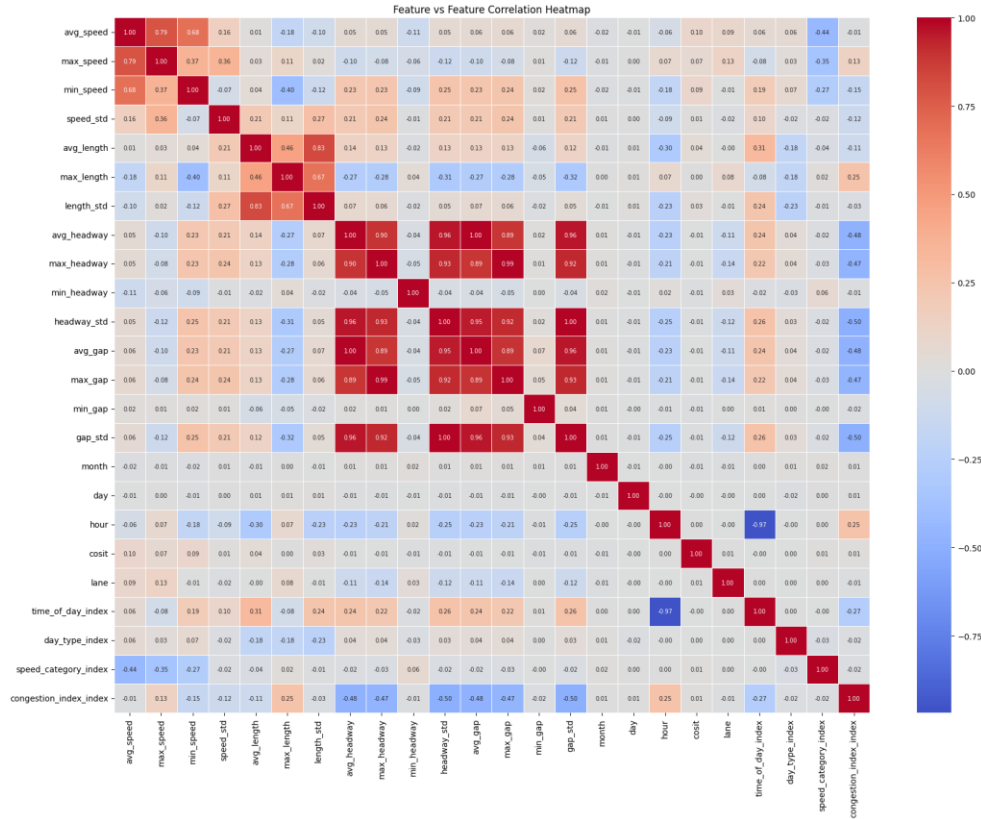


Figure 3. Feature-feature correlation heatmap for the retained candidate variables. The strongest multicollinearity appears among headway and gap statistics, while hour and time_of_day_index are near-perfect inverses.

The final modelling set contains 19 inputs: avg_speed, max_speed, min_speed, speed_std, avg_length, max_length, length_std, avg_headway, min_headway, avg_gap, min_gap, month, day, hour, cosit, lane, day_type_index, speed_category_index, and congestion_index_index. This reduced space balances predictive strength and deployability.

Variance analysis reveals that the majority of the retained features have large information content, and low-variance areas like min-headway need to be cautioned. Correlation analysis also demonstrates the importance of selection: there is a number of pairs which are highly correlated, particularly, avg_headway versus avg_gap. Instead of blindly maintaining all possible columns, the project maintains a manageable subset of the columns that is a model of the key behavioural mechanisms without undue redundancy.

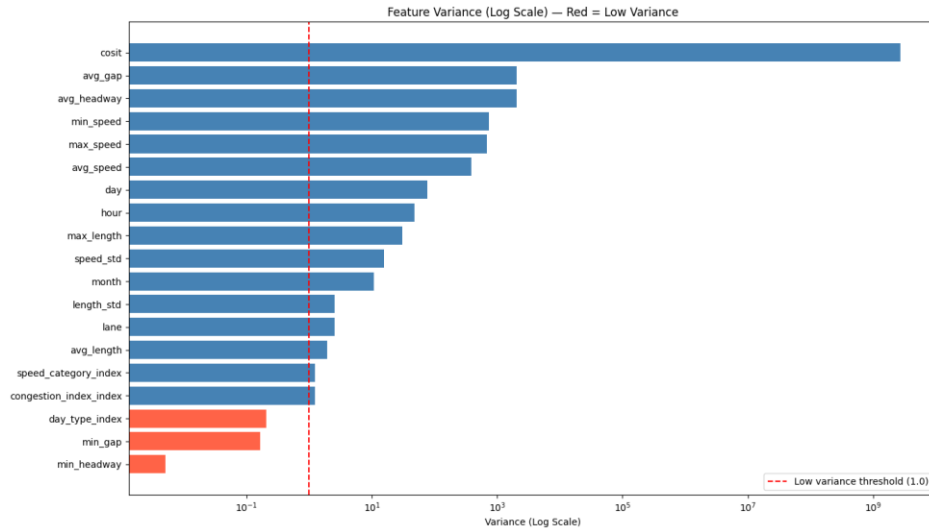


Figure 4. Feature variance on a log scale. Most retained variables preserve substantial spread, which supports stable model learning.

Although the important scores of random forests in the benchmark model are dominated by the concurrent traffic-state variables (avg_gap, avg_headway and congestion index index), it does not mean that the temporal and location variables are not essential. An explicit comparison in Notebook 3 of a benchmark model with all features against a deployable model with only pre-available inputs reveals that hour, cosit, and lane are the most significant predictors, after the removal of concurrent variables. This implies that the pre-available variables, in themselves, are statistically informative, but their signal is crowded out in the entire model by more powerful same-time operational measurements. Based on this, the following features were not only kept due to domain intuition, but also because they are the most informative set of features an actual pre-prediction deployment environment can offer.

Table 4. Final feature groups

Group	Features
Sensor measurements	avg_speed, max_speed, min_speed, speed_std, avg_length, max_length, length_std, avg_headway, min_headway, avg_gap, min_gap
Temporal	month, day, hour
Location identifiers	cosit, lane
Encoded categoricals	day_type_index, speed_category_index, congestion_index_index

8. Exploratory Data Analysis

Notebook 4 focuses on performing EDA on the final dataset traffic_features_final. This step is important because it shows that the model is not treated as a black box. Instead of jumping straight into modelling the project first builds a clear understanding of the data. This includes analysing the target variable, identifying hourly and monthly traffic patterns, examining how traffic varies by location, and exploring both individual feature distributions and their relationships with the target.

The target variable, `vehicle_count` shows a right-skewed distribution with a long tail on the higher end. This is typical for traffic data. Most hours tend to have relatively low traffic while a smaller number of periods experience very high volumes. When we look at the average hourly pattern, it becomes even more insightful. Traffic levels are lowest during the night, begin to rise quickly in the early morning, stay consistently high throughout the day and reach their peak in the late afternoon. This pattern clearly highlights that time of day is a key driver of traffic behaviour rather than just an additional feature.

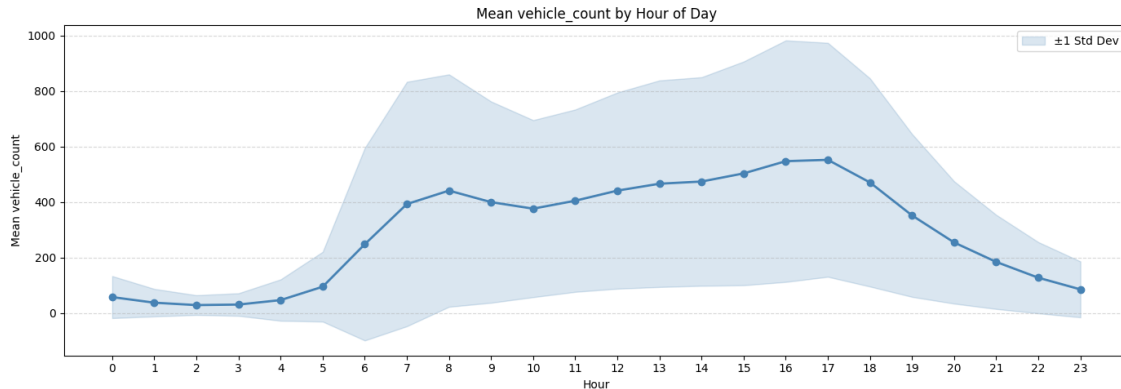


Figure 5. Mean vehicle_count by hour of day. Traffic rises sharply after early morning, remains elevated through daytime hours, and reaches its highest levels in the late afternoon.

Correlation analysis of the 19 selected features provides further insight into which variables are most strongly related to traffic volume. The strongest relationships are observed for `min_speed` (-0.4309), `avg_gap` (-0.4168), `avg_headway` (-0.4161), `max_length` (0.4046) and `lane` (0.2439). On the other hand, features such as `month` (0.0081) and `day` (0.0097) show very weak individual correlations. However, this does not mean they are useless. In nonlinear models, especially ensemble methods, features with weak individual signals can still contribute meaningfully when combined with others.

Overall, the EDA confirms that the dataset is clean, well-structured, and captures important spatial and temporal patterns. This acts as a crucial checkpoint in the workflow. It ensures that the data pipeline has produced a reliable modelling dataset before moving on to more computationally intensive steps such as baseline model training and comparison.

9. Evaluation - Baseline Regression Modelling

In Notebook 5, the final set of 19 features is combined into a single feature vector using Spark's `VectorAssembler`. The dataset is then split using a temporal approach: months 1-9 are used for training, while months 10-12 are reserved for testing. This results in 5,955,667 training rows and 1,548,246 test rows. This type of split is more appropriate than a random split because it preserves the natural time order of the data. In a time-based problem like traffic prediction, using future data to help predict the past would introduce data leakage and lead to overly optimistic results. By keeping the split chronological, the evaluation better reflects how the model would perform in a real-world forecasting scenario.

Five regression models from Spark ML are then trained and evaluated: Linear Regression, Generalized Linear Regression, Decision Tree, Random Forest, and Gradient Boosted Tree. This comparison is important because it demonstrates that model choice has a significant impact on performance. Rather than selecting a model arbitrarily, the project uses empirical results to justify the final choice.

Table 5. Baseline model comparison on the temporal test set

Model	RMSE	MAE	R2
-------	------	-----	----

Linear Regression	231.8707	166.7146	0.5740
Generalised Linear Regression	150.1206	85.1184	0.8214
Decision Tree	47.0032	19.5844	0.9825
Random Forest	43.8486	19.8082	0.9848
Gradient Boosted Tree	30.6304	15.5446	0.9926

Among all models, the Gradient Boosted Tree delivers the best performance across all evaluation metrics making it the strongest baseline. This makes it the most suitable candidate for further optimisation in the next stage.

Notebook 3 also provides an important modelling sanity check. A benchmark model using all available features achieved an R^2 of 0.9839 while a more realistic model restricted to pre-available features achieved an R^2 of 0.5105. The large difference between these results highlights the impact of removing leakage-prone features. Although performance drops, this step is necessary to ensure the model reflects a genuine forecasting scenario rather than simply reconstructing the target variable.

10. Hyperparameter Tuning and Optimisation

Notebook 6 focuses on improving the performance of the Gradient Boosted Tree regressor through hyperparameter tuning using Spark's TrainValidationSplit applied to a 30 percent sample of the training data. This approach is well suited to a big data setting as it reduces computational cost while still keeping the entire tuning process within the Spark environment. Even after sampling the tuning dataset contains 1,786,862 rows which remain very large compared to typical desktop scale machine learning tasks.

A parameter grid of 16 combinations is explored covering four key hyperparameters: maxDepth, maxIter, stepSize, and subsamplingRate each controlling different aspects of model complexity and learning behaviour. The best performing configuration identified is maxDepth 10, maxIter 100, stepSize 0.1, and subsamplingRate 1.0. A heatmap analysis of RMSE across depth and iteration further supports this result showing that deeper trees and 100 boosting iterations consistently achieve better validation performance within the tested range.

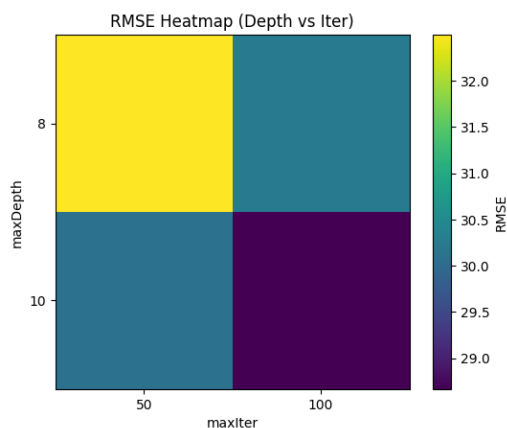


Figure 6. RMSE heatmap for the main tuning grid. The best result occurs at maxDepth 10 and maxIter 100.

Table 6. Final tuned GBT configuration

Hyperparameter	Selected value	Interpretation
maxDepth	10	Allows more expressive tree structure and captures richer traffic interactions
maxIter	100	Improves boosting strength beyond the weaker 50-iteration baseline
stepSize	0.1	Moderate learning rate balancing learning speed and stability
subsamplingRate	1.0	Uses full sampled training data during boosting

11. Results - Final Model Evaluation and Interpretation

Notebook 7 is used to re-train the final tuned GBT on the entire training set and evaluate on the withheld final quarter. The final tuned model had RMSE 27.5362, MAE 14.0627 and R^2 0.994. This represents a reduction of 2.1404 in RMSE (about 7% reduction) over the baseline GBT and an improvement in MAE.

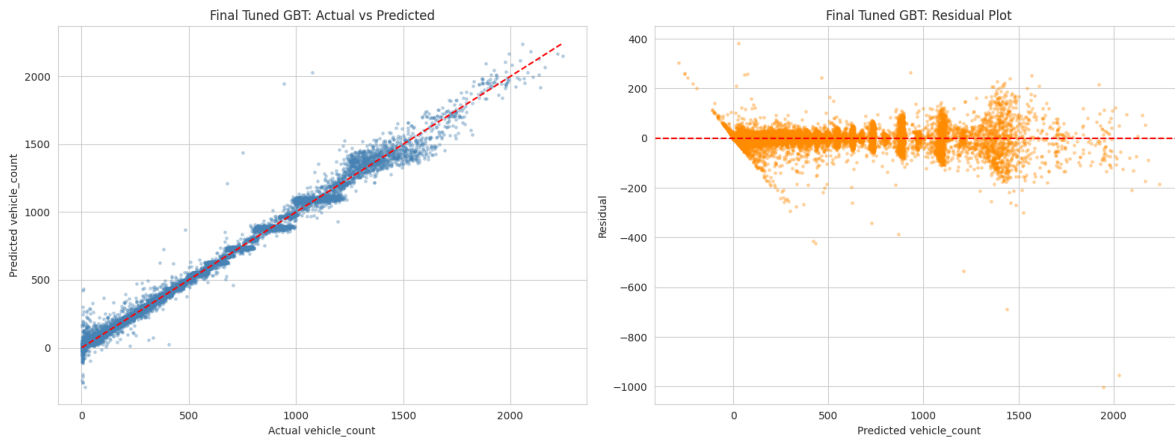


Figure 7. Final tuned GBT: actual versus predicted values and residual plot. Points align closely with the 45-degree reference line, while residuals remain concentrated around zero.

The actual vs predicted plot shows strong agreement across most of the range of the target variable. The residual plot is center close to 0 and shows greater spread during higher traffic periods, with a small number of large errors at the upper range. This is typical in traffic prediction, where higher volume periods can be affected by random events and unusual situations.

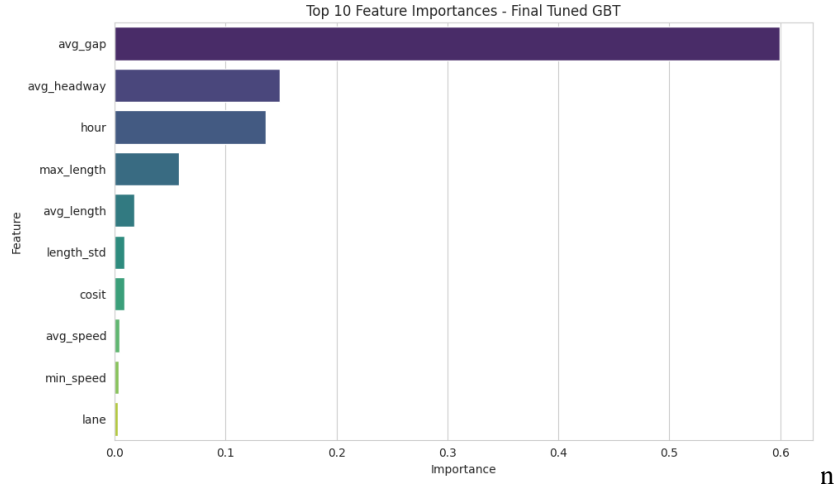


Figure 8 . Top 10 feature importances from the final tuned GBT.

The ranking for feature importance showed avg gap is by far the most important feature, followed by avg headway and hour. These variables represent the distance between vehicles and the daily pattern, and the combination of both is a strong predictor of hourly flow. Max and avg_length were also moderately important, indicating the type of traffic affects total hourly flow. Location in the form of cosit is also useful, and speed and lane made smaller contributions.

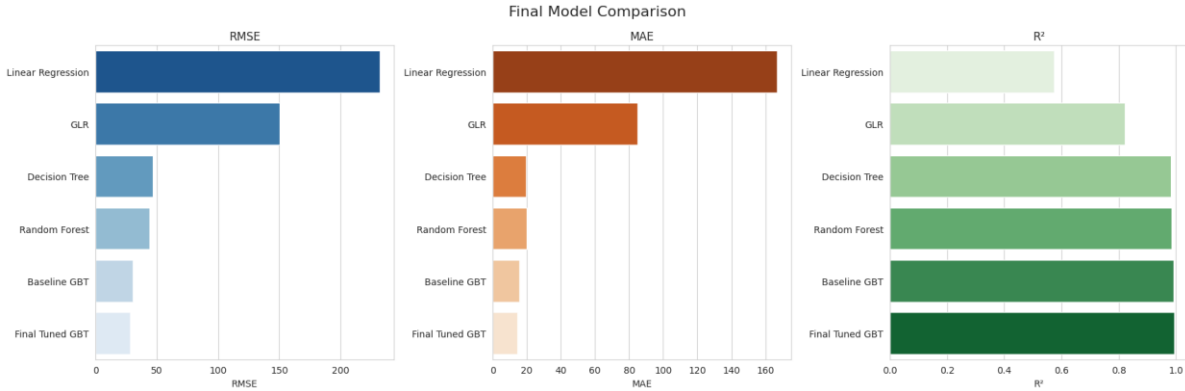


Figure 9. Comparison of all baseline models and the final tuned GBT across RMSE, MAE, and R2.

12. Discussion

One difficulty in this project was the raw nature of the TII data. The data was in per-vehicle daily files, not in a ready-to-use format for analysis, requiring an aggregation step to produce the hourly sensor-lane file. An additional issue was dealing with missing values and anomalies at the scale of this data. The missing values in the variability column came from single-vehicle hourly bins, and replacing them with 0 was a logical approach, representing no variability. Spark ML K-Means anomaly scoring was used to remove anomalous records while keeping the workflow within Spark.

The process for feature selection involved a trade-off between predictive performance and deployability, as many highly predictive variables may introduce leakage if they directly reflect target values or current traffic volumes. Finally, model selection between linear and tree-based approaches was important; linear models performed much worse, indicating that non-linear behaviour was important. It was a trade-off between prediction performance, realism, and Spark performance.

13. Limitations and Future Work

The major limitation of the predictive model is that it is based on batch-based prediction on historical data and does not operate in real time. No external features are included in the models; the only data inputs are sensor data and time information.

The system has three main directions for enhancement: firstly, it could be transformed into a structured streaming system for real-time prediction; secondly, exogenous features such as weather, holidays, events, traffic incidents, and roadworks could be incorporated; and thirdly, Spark performance could be optimised by implementing more fine-grained tuning.

14. Conclusion

In this project, an end-to-end Spark-based system was developed to predict hourly traffic volume on Irish national roads using data from 2019. The system began with raw per-vehicle data and produced a refined, feature-rich dataset suitable for modelling via scalable aggregation and feature engineering.

During the project, Spark SQL, Spark DataFrames and Spark ML were used to meet big data needs of the module. Feature selection without data leakage and a time-based train-test split are two things to ensure an appropriate and valid evaluation of the model. Gradient Boosted Tree showed to perform best of the models tested and we achieved final model that achieved an RMSE of 27.5362, an MAE of 14.0627 and R 0.994 when applied to the final test quarter data unseen previously.

Overall, this work successfully illustrates how distributed processing using Spark and machine learning techniques can be combined for meaningful analysis of traffic flow. While it operates on batch data, its strong predictive performance on a representative dataset forms a good foundation for more advanced real-time prediction systems.

15. References

- [1] B. Chambers and M. Zaharia, *Spark: The Definitive Guide*. Sebastopol, CA, USA: O'Reilly Media, 2018.
- [2] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI)*, 2004, pp. 137–150.
- [3] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2012, pp. 15–28.